

Teaching Gemma 3 (1B) to Reason with GRPO

Workshop 3 - AI

Thomas Martinod¹

¹Ingeniería Física, Universidad EAFIT, Colombia, tmartinods@eafit.edu.co

21 de noviembre de 2025

1. Introduction

This workshop explores how reinforcement learning from verifiable feedback can improve the reasoning abilities of large language models. Specifically, we fine-tune the **Gemma 3 (1B)** model using the **Group Relative Policy Optimization (GRPO)** algorithm, an efficient alternative to PPO within RLHF-style pipelines. The purpose is to teach the model not only to choose correct answers in multiple-choice reasoning tasks, but also to generate coherent step-by-step reasoning before selecting its final answer.

The goals of the workshop are threefold. First, we aim to understand the structure and reasoning requirements of multiple-choice QA datasets such as RACE or QuALITY. Second, we design and implement a verifiable reward function capable of checking whether the model's predicted answer matches the gold label. Third, we fine-tune the Gemma 3 model using GRPO and compare its performance against the base model in order to evaluate improvements in reasoning accuracy and output quality.

The official instructions for this workshop are available in the course repository at [this link](#). All experiments, trained models, notebooks, and results produced for this project are stored in the author's personal repository [thomas-martinod/ai-workshops](https://github.com/thomas-martinod/ai-workshops).

This report presents the methodology, implementation details, and outcomes of the fine-tuning process, including both quantitative measures of performance and qualitative examples showing the evolution of the model's reasoning abilities.

2. Dataset Exploration and Base Model Evaluation

In the first part of the workshop, we explored the structure of the RACE dataset and evaluated the baseline performance of the `unsloth/gemma-3-1b-it` model. The dataset consists of short passages followed by a question and four answer options (A–D). Each item requires some level of reasoning, such as inference, explanation, or understanding of contextual clues, which makes the dataset appropriate for verifiable reinforcement learning.

After loading the base model with Unsloth's optimized Gemma integration, we experimented with two prompt styles. The first simply asked the model to choose an answer. The second encouraged a short reasoning process by telling the model to “think step-by-step before answering.” Both formats were applied to a small subset of examples taken from the validation split.

To evaluate performance, we used a simple rule-based scoring function that extracts the predicted answer letter and compares it with the correct label. On the small evaluation subset (ten examples), the base model answered none of the questions correctly, giving an accuracy of 0 %. While this is a very small sample, it clearly shows that the model struggles with the type of reasoning required by RACE.

Qualitatively, prompt design had an effect on how the model expressed its thoughts, but not on correctness. The chain-of-thought prompt produced longer and more structured responses, but the final answers remained incorrect. This confirms that although prompting can change the style of the model's output, it does not necessarily improve reasoning accuracy without additional alignment or fine-tuning.

These observations serve as a clear starting point for the GRPO training stage, where we aim to help the model improve both its reasoning process and its final answer selection.

3. Fine-Tuning with GRPO

In the second stage, we fine-tuned the base `unsloth/gemma-3-1b-it` model using the GRPO algorithm. GRPO is a reinforcement learning method that compares several generated answers for each prompt and updates the model according to the relative rewards. Because the RACE dataset contains verifiable multiple-choice labels, we used a simple rule-based reward instead of training a separate reward model.

3.1. Reward Function

We implemented a small function called `mc_reward` that checks whether the model's final answer letter (A–D) matches the gold label (correct answer). From the generated text, the function extracts the last valid letter it finds and assigns:

$$\text{reward} = \begin{cases} +1, & \text{if prediction} = \text{gold} \\ -0.5, & \text{if prediction} \neq \text{gold but still an answer A, B, C, or D} \\ -1, & \text{otherwise} \end{cases}$$

If the model does not produce a letter at all, the reward defaults to -1 . This creates a fully verifiable feedback signal suitable for GRPO, allowing the model to learn from correctness without human annotations.

3.2. Training Configuration

We fine-tuned the model using:

- learning rate: `5e-6`
- 4 generations per prompt
- maximum completion length: 64 tokens
- GRPO KL coefficient: 0.02
- total steps: 50

The full training loop ran for approximately 15 minutes on Google Colab. We intentionally reduced the training size (50 GRPO steps instead of 10,000) and shortened the output length to avoid the original 40-hour runtime implied by the default configuration.

3.3. Model Upload and Quick Evaluation

After training, we uploaded the resulting model to the Hugging Face Hub:

<https://huggingface.co/th-martinod/gemma-3-grpo>

To test the model, we sampled unseen examples from `train[10000:20000]` of the RACE dataset and computed the reward. A representative example is:

Correct answer: C

Obtained reward: $-0,5$

Although the model did not produce the correct final choice in this sample, the test confirms that the fine-tuned model generates structured answers compatible with the reward function and completes the GRPO pipeline successfully.

In summary, this stage implemented a full reinforcement learning loop with GRPO: prompt generation, reward computation, policy updates, model upload, and evaluation. The reduced configuration allowed us to complete RL fine-tuning efficiently within Colab limits while preserving the intended reasoning workflow.

4. Evaluation and Comparison

In the final stage of the workshop, we compared the performance of the base `unsloth/gemma-3-1b-it` model with the fine-tuned GRPO version. The goal was to evaluate whether reinforcement learning improved the model’s ability to select the correct option in multiple-choice reasoning questions. To ensure fairness, both models were evaluated on the same subset of 30 examples from the RACE dataset using the same prompt format, but using a simpler evaluation reward.

4.1. Accuracy Results

Because GRPO relies on relative rewards across several generated completions, the reward used during training was not identical to the reward used during evaluation. During fine-tuning, GRPO compared the four sampled completions and assigned rewards based on their relative correctness, sometimes producing values such as $+1$, $-0,5$, or -1 . In contrast, for evaluation we used a strict correctness metric: the model received a score of 1 if it produced the correct letter and 0 otherwise. This difference is standard in reinforcement learning workflows, where training rewards shape the policy while evaluation rewards measure accuracy directly.

The resulting accuracy for both models is shown below:

Model	Accuracy
Base model	0.47
Fine-tuned GRPO model	0.47

The two models obtained the same accuracy on this subset. This outcome is expected, as the GRPO training was intentionally reduced to 50 steps to keep the runtime reasonable in Google Colab. With such a short training duration, the model does not yet accumulate enough updates to significantly change its behavior.

4.2. Qualitative Comparison

To complement the numerical evaluation, we inspected three examples from the dataset. In all cases, both models produced well-structured outputs consistent with the expected format (a brief reasoning followed by a final letter). The reasoning steps differed slightly between the base and fine-tuned versions, but the final predictions were often similar, which aligns with the limited amount of fine-tuning.

4.3. Reflection

Even though the accuracy did not improve after GRPO fine-tuning, the experiment successfully demonstrated the full reinforcement learning pipeline with verifiable feedback: prompt construction, reward computation, GRPO optimization, model upload, and model evaluation. The fine-tuned model interacted correctly with the reward structure and preserved output formatting. With more GRPO steps, longer completions, or a smoother reward design, improvements in reasoning performance would likely appear.

This workshop demonstrated how reinforcement learning with verifiable feedback can be applied to improve (or at least not worsen) reasoning-oriented language models. Using GRPO, we implemented a complete RL pipeline: dataset preparation, reward design, fine-tuning, model uploading, and comparative evaluation. Although the fine-tuned model did not outperform the base model within the limited 50 training steps, the experiment confirmed that the model learned to interact correctly with the reward structure and preserved the expected output format. The results highlight that meaningful improvements would require longer training or richer reward shaping, but the core methodology was successfully implemented.